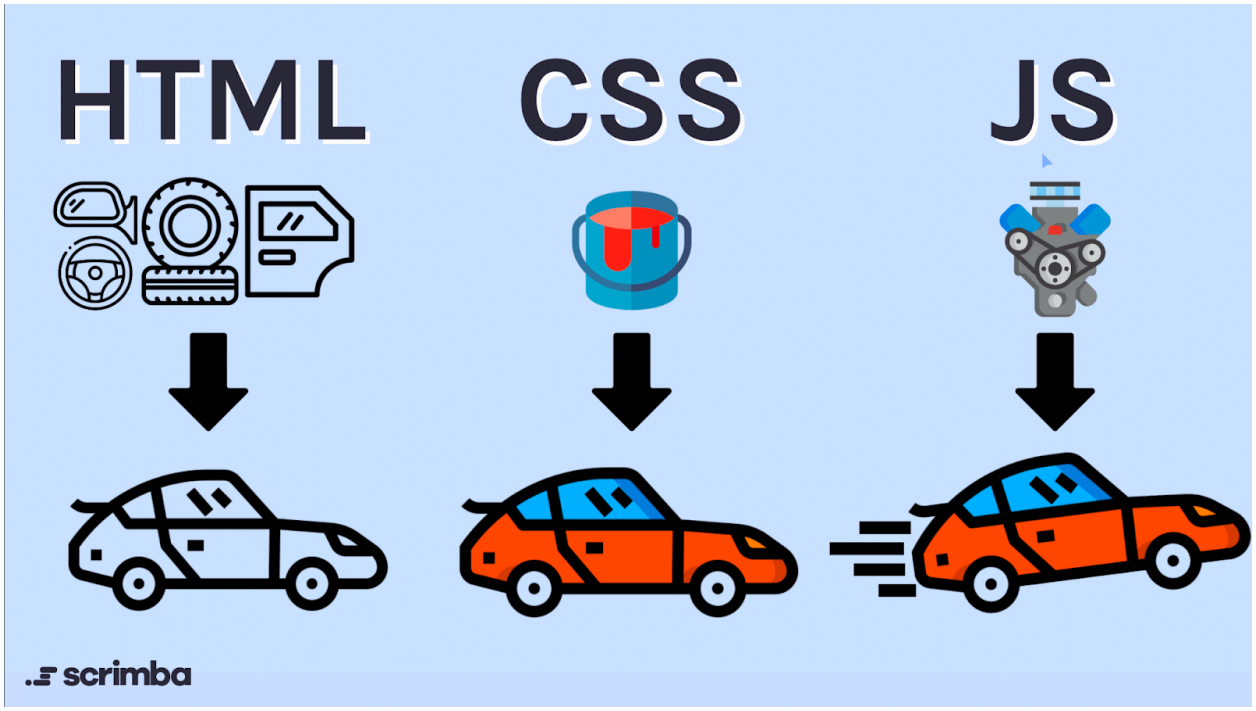
Java Script ⇒ .js ⇒ .ts

It is Programming scripting language

It give interactive ness or Functionalities of Web Pages

Java script is a high level programming scripting language and it's give more interactiveness of the web pages and web application.

We create or develop a static web application using only html,css when will you add your scripting concept of js that time only project will transforming to Dynamic web application



Importing methods 😎

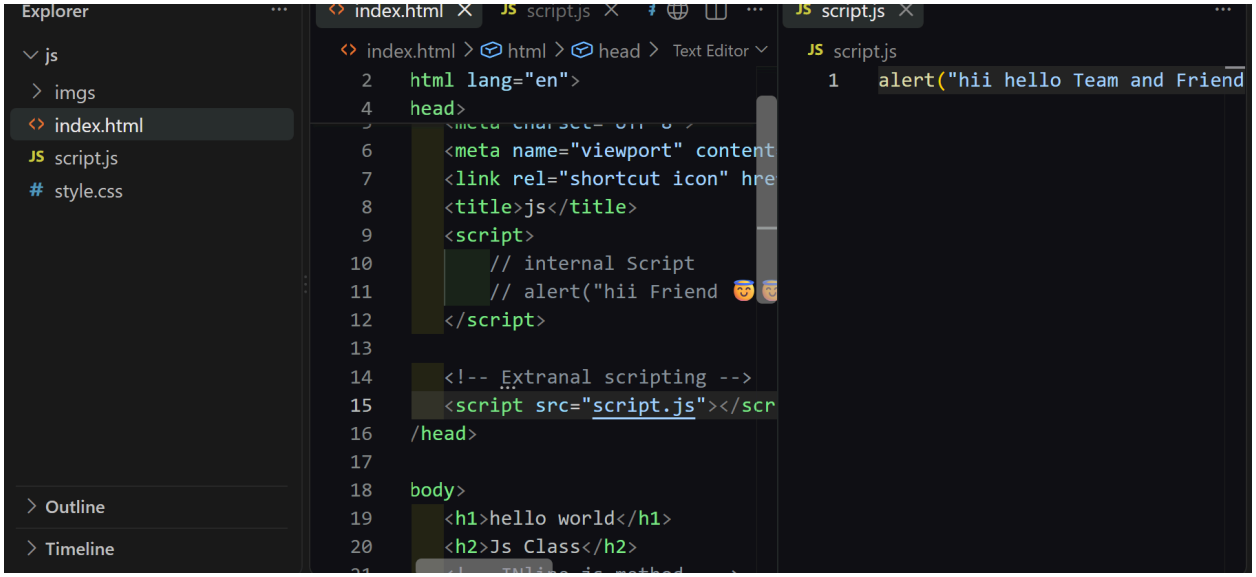
Inline Scripting method

```
<body>
    <h1>hello world</h1>
    <h2>Js Class</h2>
    <!-- INline js method  -->
    <script>
        alert("hii team")
    </script>
</body>
```

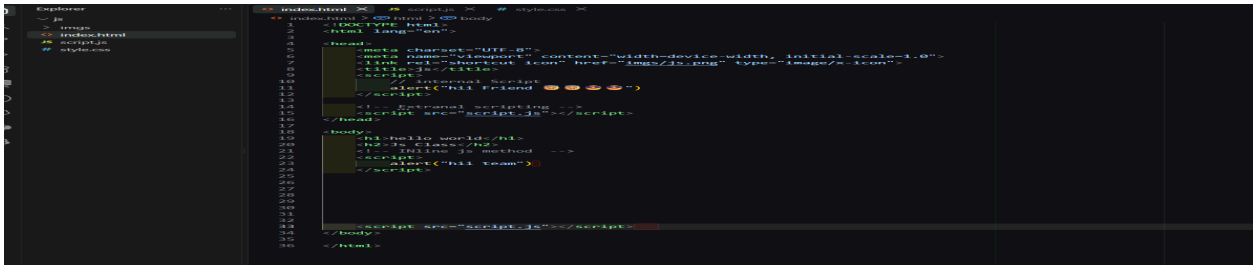
Internal Scripting method

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="shortcut icon" href="imgs/js.png" type="image/x-icon">
    <title>js</title>
    <script>
        // internal Script
        alert("hii Friend 😇😇😎😎")
    </script>
</head>
```

External Scripting method



LExical Scripting method



Javascript Vs Java 👍

Java	Javascript
Core Language	Scripting language
Combination of OOPS & DSA	Optional ⇒ OOPS & DSA
Structure & Syntax or Rules we wont avoid or overcome	Structure & Syntax or Rules we will or can avoid or overcome
Backend Logics	Frontend Logic or validation
Java ⇒ oops dsa ⇒ core concepts	ECMA script ⇒ similar to java ⇒ Js Js ⇒ ECMA script 5(ES 5) , ECMA script 6 (ES 6) ⇒ es6 (js)

JS Concepts 👍

ES 5 Topics

Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
Data Type ⇒ 2 +1

Primitive Data 7	Non primitive Data 3
Number String Boolean Bigint Undefined Null Symboles	Array ⇒ =[], ⇒ = [‘ ‘,””,1,true] Object ⇒ ={}, ⇒ ={ key : value } Function

```
<script>
        //  Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Data Type  ⇒ 2 +1
        // Stractured Data type  ==> Primitive Data type (7)
        // Number  ==> [0,1,2,3,4.......n] ==> without using any
symblole  "",''
        let num1 = 100;
        const Total = 2000;
        let myNum = 10.80;
        console.log(num1);
        console.log(Total);
```

```
        console.log(myNum);
        // String ==> [a,b,c,d......z] ==> with using any symbole like "
",''
        let myName = " ganapathi";
        let myName1 = 'raja';
        console.log(myName);
        console.log(myName1);
        // Boolean ==> [true, false ==> 0,1]
        // let isValid = true;
        let isValid = false;
        console.log(isValid);
        // bigINt [ (2n*3) ==> more the 5 digits ]
        let myMobile = 75202132895
        let myValue = 100000000000000000000000
        BigInt(myMobile);
        BigInt(myValue);
        console.log(BigInt(myValue))
        // Undefined ==> [ ? ] ==> may be getting a values
        var a = 1;
        console.log(a);
        // Null ==> [ { } ]  ==> never getting a values
        var a = null;
        // symbole [ @,!,#,$,%];

        // UnStractured Data type  ==> NON - Primitive Data type (3)
    </script>
```

```
<script>

        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Array  ==> arr or [ '',"",10]
        // Hytrogenius element an dstore continuos=use loaction
        // it has two important key words
        //  index ==>start with 0==>  addresss ==> position ==> refernce
key,
        //  element ==> values ==> datas ==>
        var myArr = [100, 200, 300, "ganapathi", "sunder", "hari", true,
false, 100.200, "%^$", 7502132895, "pc", "hello", '1'];
```

```
        console.log(myArr);
        console.log(myArr[0]); // index finding
        console.log(myArr[12]);
        // console.log(myArr[13]);
        // console.log(myArr[8]);
        // console.log(myArr[8, 9, 10]);

        // Object  ==> Stand alone entity  ==> obj or  = {} or ={key :
value}

        var myCar = {
            name: "spressoo",
            color: "grey",
            price: 500000
        };

        var myCar1 = {
            name: "spressoo",
            color: "grey",
            price: 500000,
            quality: function functionality() {
                console.log(myCar1.name);
                console.log(this.color); // 3rd calling object value ==>
this lkeyword to cll inside object within the function
            }
        };
        console.log(myCar);
        console.log(myCar.name); //1st method ==>  object litral method
        console.log(myCar.color);
        console.log(`${myCar.price}`); //2nd method ==>  object
destructing method

        console.log(myCar1.quality())
        //  console.log(${ myCar.price }); // object destructing method

        // Function  ==> fnction ==> funName () { return } ==>
=()=>{return}
    </script>
```

```
<script>
        //  Array  ==> = [ ]

        let myDatas = [1, 2, 3, 4, 5, "a", "b", "c", true, false, null,
7502132896,];
        console.log(myDatas);
        console.log(myDatas[0]);
        console.log(myDatas[1]);
        console.log(myDatas[5]);

        console.log(myDatas.indexOf(true));

        // Object ==> ={ key : value  }

        const car = {
            name: "bmw",
            color: "navy",
            price: 1200000,
            model: "m5"
        }

        console.log(car)
        console.log(car.model)
        console.log(car.name) // . 1st method litral

        console.log(`${car.price}`)
        console.log(`${car.color}`) // `${}` 2nd method object
desctructing

        let waterBottle = {
            name: "jio",
            qty: "5l",
            price: 20,
            capacity: function size() {
                let smallSize = "250ml";
                let sample = this.name
                console.log(smallSize)
                console.log(waterBottle.qty)
                console.log(this.name) // this 3rd method (.this) method
```

```
                console.log(this.price) // this measn value assigner
            }
        };
        console.log(waterBottle)
        console.log(waterBottle.capacity())
        // console.log(this.AnimationTimeline) // not adviceable

        // Functions  ==> (){} or =()=>{} ==> or function
        // Reusable code (organized) collection to perform single loci
operations
        // function funName(){ return } or const funNAme =()=>{return};

        // console.log(100 + 200);



        // function ==> 2 classification
        // with Parameter function
        // parameter ==> arguments ==> reference datas  ==> props ==>
params
        function sumOftwoNumbers(a, b) { // a,b parameter ==> value
carrier
            let c = a + b;
            return c;
        }

        const sumOfNumbers = (value1, value2) => {
            return valuesAdd = value1 + value2;
        }

        console.log("nasika 100,200:  ", sumOftwoNumbers(100, 200));
        console.log("pavidra 1000,2000:  ", sumOftwoNumbers(1000,
2000));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 20));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 1));

        // without  Parameter function

        function Nasika() {
            let a = 100;
```

```
            var b = 300;
            const c = a + b;
            return c;
            console.log(c)
        };
        Nasika();
        console.log(Nasika());

        function Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        };
        Pavidra()

        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())


        // functions 4 types
        // Function expression method
        // function FunName () { return }
        function Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        };
        Pavidra()
        // Arrow method ==> HOF ==> Call back ==> recurssive function
==> cohansive function ==>
        // variabledeg funcName =()=>{ return }
        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())
```

```
        // IIFE
        const myMainfun = (
            function Pavidra() {
                let x = 200, y = 300;
                const z = x + y;
                console.log(z);
            }
        )();
        // Generative Method
            function* Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        }

    </script>
```

Variable ⇒ Containers (temp storage) ⇒ 3 ⇒ var ,let, const
Variable Scoping & Hoisting ⇒ { } ⇒ 3

```
<script>

        // Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Collection of Inforamtaions or detalis
        // Raw facts ==> 0,1 ==> true  false
        // Data type ==> 2
        // Stractured Data type  ==> Primitive Data type (7)
        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Variable  ⇒ Containers (temp storage ) ⇒ 3 ⇒ var ,let,
const
        // var let const
        // To store temporarly in data in Opertaion basis

        // a = "ganapathi";

        // var a = "ganapathi";
        // var a = "sundar";
```

```
        // let a = "Keerthi";
        // let a = 1000;

        // const pi = 3.14
        // const Opi = 22 / 7

        // let pi = 20;

        // const a = 100;
        // const a = "ganapathi";

        // variable  varName = values

        var obj1 = "hii"
        let obj2 = 'hello'
        const obj3 = 100;

        // Variable  ⇒ {  } == > Scoping => 3
        // global scope  ==> ES5 scope  ==> var let const
        // local  scope ==>  Block level scope ==> functional scope ==>
let const
        // lexical  scope  ==> let const


        var trainer = "ganapathi";
        function batch14() {  // Globle scope
            console.log("hii globel", trainer);
            let trainee = "pavidra";
            console.log(trainee);

            function team1() { // local scope
                console.log(trainer); // lexivcaln
                console.log(trainee);
                const trainee1 = "Mani"
                console.log(trainee1)
            }
            team1()
            console.log(trainee1)
```

```
    }

    console.log(trainer);
    batch14();
    // console.log(trainee);


    function sla() {
        var trainer = "ganapathi";
        function batch14() {
            console.log(trainer);
            let trainee = "pavidra";
            function team1() {
                console.log(trainer);
                console.log(trainee);
                const trainee1 = "Mani"
                console.log(trainee1)
            }
        }
    }

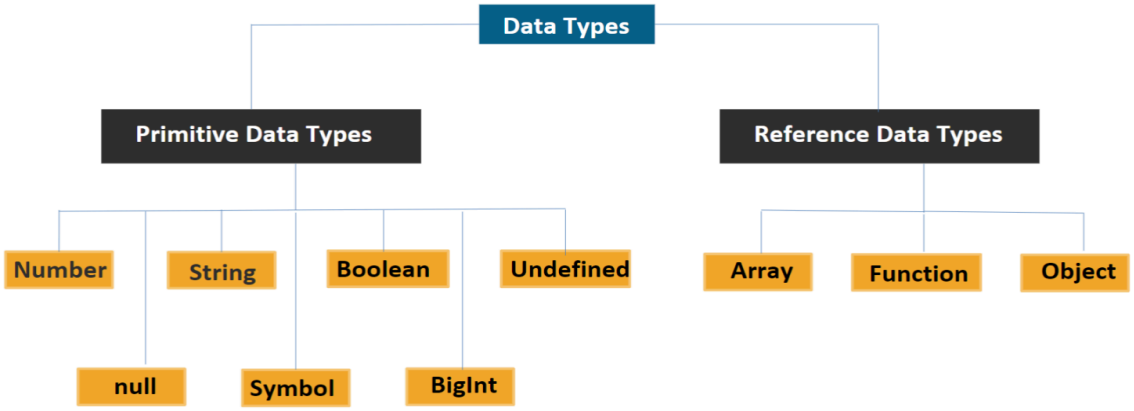
</script>
```

Var	Let	Const
Es5 version variable	Es6 version variable	Es6 version variable
Void scoping (public)	Block / functional scoping	Block / functional scoping
Global scoping variable	Local scoping variable	Local scoping variable
Read / write operation can perform	Read / write operation can perform	Read operation can perform
Changeable / Mutable	Changeable / Mutable	Non Changeable / imMutable
Scoping means ⇒ { }		
Variable Hoisting ⇒ var		

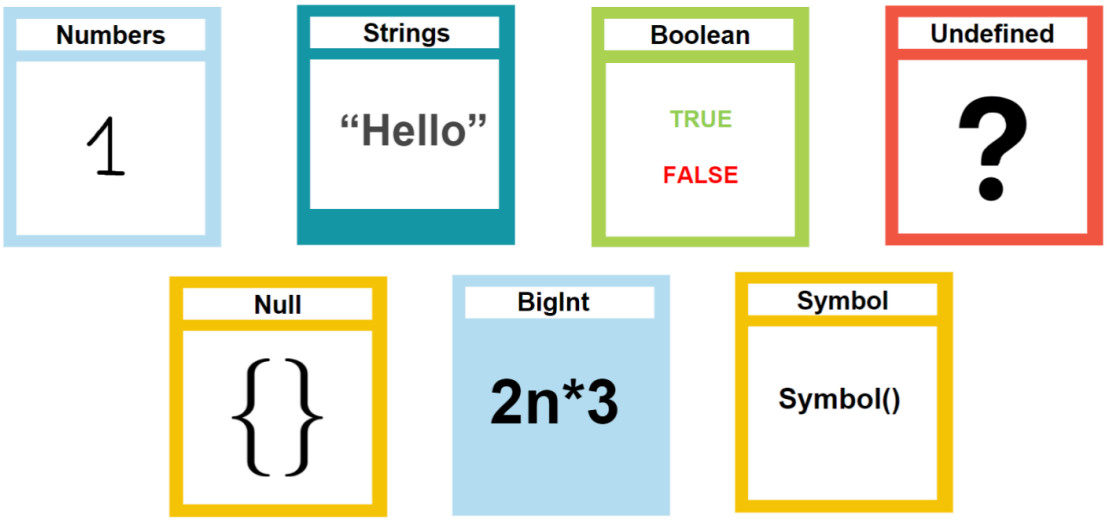
Web hoisting ⇒ Local project will migrate globalize project

```
// Variable Hoisting
        // const a = "Ganpathi";
        // let a = "Ganpathi";
        // var a = "Ganpathi";
        console.log(a);
        var a = "Ganpathi Raja";
        // let a = "Ganpathi Raja";
        // const a = "Ganpathi Raja";
```

Datatypes in JavaScript



Primitive Data Types



Reference Data Types

Objects: An object is a standalone entity. It consists of properties and methods.

```
Syntax: let obj1 = { }
```

Arrays: An Array is a collection of heterogenous elements which are stored at contiguous memory locations.

```
Syntax: let arr[ ] ;
let arr = new Array( )
```

Functions: A function is a block of organized, reusable code that is used to perform a single, related actions.

```
Syntax: function
name() {
alert("This is a
function");
}
```

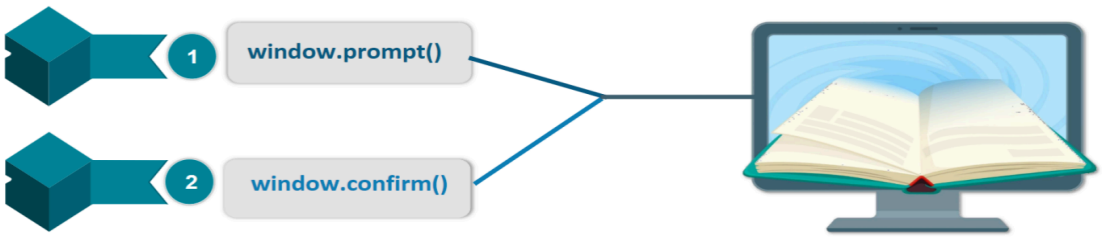
Understanding JavaScript Output Statements

- In JavaScript, there are different ways to display the data.
- Some of the commonly used ways to display output are:

Statement	Description
innerHTML	It writes a data into a certain HTML element
document.write()	It displays a data in the HTML output
window.alert()	It displays the data in the alert box
console.log()	It displays the data in the browser console

Understanding JavaScript Input Statements

- It is important to take input from the user to display the data as per their need.
- Apart from forms, JavaScript provides two statements to accept input from the user as mentioned below:



Js I/O Methods

Input method ⇒ 2

Output Method ⇒ 4

```
<h2>Output methods</h2>

    <button onclick="outputMethod()">click me see my output</button>

    <h2 id="outputmethod">hii output methods</h2>

    <button onclick="outInnerhtmlmethod()">click me InnerHtml</button>



    <div>
        <h2 id="input1"></h2>
        <h2 id="input2"></h2>
        <button onclick="inputMethod1()">click Give input 1</button>
        <button onclick="inputMethod2()">click Give input 2</button>
    </div>
    <script>
        // Inputs 2 /
        // window.prompt();
        // window.prompt("Enter your thougt today ...!")
        function inputMethod1() {
            const userMsg = window.prompt("Enter your thougt today ...!");
            let myThought = document.getElementById("input1");
            myThought.innerHTML = userMsg;
        };
        // window.confirm();
        function inputMethod2() {
            if (window.confirm("Are you sure submit your form..👍👍!")) {
                let myinput2method = "successfull submitted your
form...👍👍👍"
                let myThought = document.getElementById("input2");
                myThought.innerHTML = myinput2method;
            }
            else {
                let myMessages = "Your Application not submitted ..sorry
....❌❌🤦‍♂️🤦‍♂️";
                let myThought = document.getElementById("input2");
                myThought.innerHTML = myMessages;
```

```
        }

    };



    //  Output methods 4
    // console.log(); ==> 4
    const myName = "Ganapathi Raja ";
    console.log(myName);
    console.info(myName);
    console.warn(myName);
    console.error(myName);


    // window.alert() ==> 2
    // window.alert(myName);
    // alert(myName);


    // document.write()
    document.write(myName);
    function outputMethod() {
        let messages = "hii team and Friends...😍😎😎😇😂";
        // document.write(messages);
        document.writeln(messages);
    };
    // innerHTML
    const outInnerhtmlmethod = () => {
        let messages = "hii team and Friends...😍😎😎😇😂";
        let myPlacehtml = document.getElementById("outputmethod");
        myPlacehtml.innerHTML = messages;
    }
</script>
```

Operators ⇒ 4 + 2

Z = x+y ⇒ let z = a+b;

Operands ⇒ x,y,z

Operators ⇒ +.=

OPeration ⇒ addition
Variable ⇒ let ⇒ temp

Operators	Symbols or Operators
Arithmetic Operators (7)	+,-,*,/,%,++,--
Assignment Operators (8)	=,=+,=-=,*,=/,=%,
Comparison Operators	==,===,!=,<,=<,>,>=
Logical Operators (3)	AND &&,OR ||,NOT !
Ternary ⇒ ?	? ⇒ if statements
typeOf ⇒ Operators	typeOf()

```
<script>



        // Arithmetic Operators (7)
        // +,-,*,/,%,++,--
        let a = 10; b = 30;
        const key = 1;
        // const key = 2;
        // const key = 3;
        // const key = 4;
        // const key = 5;
        // const key = 6;
        // const key = 7;
        // const key = 17;
        switch (key) {
            case 1:
                var c = a + b;
                console.log(c);
                break;

            case 2:
                var c = a - b;
                console.log(c);
                break;
```

```
        case 3:
            var c = a * b;
            console.log(c);
            break;
        case 4:
            var c = a / b;
            console.log(c);
            break;
        case 5:
            var c = a % b;
            console.log(c);
            break;
        case 6:
            a++
            var c = a + b;
            console.log(c);
            break;
        case 7:
            a--;
            var c = a + b;
            console.log(c);
            break;
        default:
            console.log("You enter invalid Operation")
            break;
    }


    // let x = 200; y = 400; z = x + y;
    // console.log(z);

    // console.log(10 + 20);

    // Assignment Operators (8)
    // =,=+,=-=,*,=/,=%,

    let x = 200;
    let y = 400;
    // x++;
```

```
x = x + 40;
y = y - 70;
y = y * 2
const z = x + y;
console.log(z);

// Comparison Operators
// ==,===,!=,<,=<,>,>=

let x = 200;
let y = 400;

if (x > y) {
    console.log("X is bigger value", x)
}
else {
    console.log("Y is bigger value", y)
}

let value1 = 100;
const value2 = 100;
const value2 = 1000;
const value2 = "1000";
const value2 = true;
= , == , ===
if (value1 = value2) {  // assigneing the values
if (value1 == value2) { // check the values are same or not
if (value1 === value2) { // vaues and their data type
if (!(value1 === value2)) { // vaues and their data type
    console.log("values same")
}
else {
    console.log("values not same")
}

// Logical Operators(3)
// AND &&, OR ||, NOT!

let value1 = 1000; value2 = 2000; value3 = 400;
```

```
if ((value1 > value2) && (value1 > value3)) {
    console.log("Value 1 is bigger", value1)
}
else if ((value2 > value1) && (value2 > value3)) {
    console.log("Value 2 is bigger", value2)
}
else {
    console.log("Value 3 is bigger", value3)
}

if ((value1 > value2) || (value1 > value3)) {
    console.log("Value 1 is bigger", value1)
}
else if ((value2 > value1) || (value2 > value3)) {
    console.log("Value 2 is bigger", value2)
}
else {
    console.log("Value 3 is bigger", value3)
}



//typeOf operator
let ganapathi = 100;
let myname = "ganapathi";
let isvalidate = true;
let isvalidated = false;
let myvalue =

    function add() {
        return
    }

const hi = () => {
    return
}

let myarry = {
```

```
            hi: 100,
        };

        let myObject = [1, 2, 4, 5];

        console.log(typeof (ganapathi));
        console.log(typeof (myname));
        console.log(typeof (isvalidate));
        console.log(typeof (isvalidated));
        console.log(typeof (myvalue));
        console.log(typeof (funame));
        console.log(typeof (add));
        console.log(typeof (hi));
        console.log(typeof (myarry));

        console.log(typeof (myObject));
        console.log(Array.isArray(myObject));

        //ternary operator ==> ?

        let ganapthi = 12;
        const voting = 18;
        // if (ganapthi >= voting) {
        //     console.log("eligible to vote");
        // }
        // else {
        //     console.log("not eligible to vote ");
        // }

        console.log(ganapthi >= voting ? "eligible to vote" : "not
eligible to vote");

    </script>
```

Statements ⇒ 2

Conditional statements (4)
Looping Statements (5)

https://pythontutor.com/visualize.html

Conditional statements (4)	Looping Statements (5)
if(){ Statements }; **if(){ Statements } else{ Statements};** **if(){ Statements }** **else if(){ Statements }** **else{ Statements};** **switch(){** **Case: “case”** **Break** **…** **Default:** **}**	**for(){ Statements };** **for(of) { Statements };** **for(in) { Statements };** **While () { Statements };** **do{ Statements } while();**

```
<script>
        //Conditional statements (4)
        // if(){ Statements };  ==> true (max) ==> success
        // let todaClass = true;
        let todaClass = false;
        if (todaClass) {
            alert("Yes, Definiy classes are there!.....")
        }



        let votingEligiblity = 18;
        // const ganapathi = 17;
        const ganapathi = 26;
        if (ganapathi > votingEligiblity) {
```

```
            alert("Eligible to voting ....!😍😍😍😍");
        }

        // if(){ Statements } ==> max (true, success) else{ Statements};
==> max (false, failure)
        if (todaClass) {
            alert("Yes, Definiy classes are there!.....")
        }
        else {
            alert("No,  classes today!.....")
        }



        if (ganapathi > votingEligiblity) {
            alert("Eligible to voting ....!😍😍😍😍");
        }
        else {
            alert("Not Eligible to voting ....!❌❌🤦‍♂️🤦‍♂️");
        }

        // if(){ Statements }
        // else if(){ Statements }
        // else{ Statements};  ==> nested if statements ==> ladder

        let a = 1000; b = 2000; c = 3000; d=4000;
        // if (a > b) {
        //     alert("A is bigger value..!")
        // }
        // else if (a > c) {
        //     alert("A is bigger value..!")
        // }
        // else if (b > a) {
        //     alert("B is bigger value..!")
        // }
        // else if (b > c) {
        //     alert("B is bigger value..!");
        // }
        // else {
        //     alert("C is bigger value..!")
```

```
// }

if ((a > b) && (a > c)) {
    alert("A is bigger value..!")
}
else if ((b > a) && (b > c)) {
    alert("B is bigger value..!")
}
else {
    alert("C is bigger value..!")
}

// switch(){
// Case: “case”
// Break
// …

// Default:
// }


// let userName = "superAdmin";
// let adminPass = "aa@123";

// if (userName == "superAdminmm") {
//     console.log("login completed...")
//     if (adminPass == "aa@123") {
//         console.log("login completed...");
//         console.log("Valid PAssword...");
//     }
//     else {
//         console.log("in Valid PAssword...");
//     }
// }
// else {
//     console.log("Can't login");
// }

// switch(){
```

```
// Case: “case”
// Break
// …

// Default:
// }

// arithmetic Operation

let a = 100; b = 200;
// let key = "add";
// let key = "sub";
let key = "divhjjhj";
switch (key) {
    case "add":
        let add = a + b;
        console.log("addition: ", add);
        break;

    case "sub":
        let sub = a - b;
        console.log("addition: ", sub);
        break;

    case "multi":
        let multi = a * b;
        console.log("addition: ", multi);
        break;

    case "div":
        let div = a / b;
        console.log("addition: ", div);
        break;
    case "modulo":
        let modulo = a % b;
        console.log("addition: ", modulo);
        break;
    case "Inc":
        a++;
```

```
                let Inc = a + b;
                console.log("addition: ", Inc);
                break;
            case "Dec":
                b--;
                let Dec = a - b;
                console.log("addition: ", Dec);
                break;

            default:
                console.log("You enter a wrong operation")
                break;

            // finally:
        }



    </script>
```

```
<script>
        // Looping Statements (5)
        //init
        //condition checking
        //inc/ dec

        // for(){ Statements };
        // for (let i = 0; i <= 5; i++) {
        //     console.log(i);
        // }
        // let i = 0;
        for (let i = 0; i <= 5; i++) {
        //     console.log(i);
        // }

        // let i = 0 ==> 0<=5 ==> console.log(i); ==> 0++ = 1
        // let i = 1 ==> 1<=5 ==> console.log(i); ==> 1++ = 2

        // let i = 4 ==> 4<=5 ==> console.log(i); ==> 4++ = 5
```

```
        // let i = 5 ==> 5<=5 ==> console.log(i); ==> 5++ =6
        // let i = 6 ==> 6<=5 ==> x


        // for(of) { Statements };

        let myArr = [10000, 2, 3, 4, 5, 6, 7];
        console.log(myArr[0]);
        console.log(myArr[1]);
        console.log(myArr[3]);



        for (index of myArr) {
            console.log(index)
        }

        // for(in) { Statements };

        let myObj = {
            name: "ganapathi",
            mobile: 7502132895,
            gmail: "vgr@gmail.com"
        };
        console.log(myObj);
        console.log(myObj.name);
        console.log(myObj.mobile);
        console.log(myObj.gmail);

        for (keys in myObj) {
            console.log(`my object value ${keys} and values
${myObj[keys]}`)
        }


        // While () { Statements };
        let i = 1;
        while (i < 6) {
            console.log("while loop", i);
```

```
            i++;
        }

        // let i = 1
        // 1<=6;
        // console.log("while loop", i);  ==> 1 ++

        // do{ Statements } while();

        let i = 0;
        do {
            console.log("while loop", i);
            i++;
        } while (i <= 5);


    </script>
```

INbuilt Object Function

Math object (5)

Date Object (5 to 8)

```
<script>
        // INbuilt Object Function
        // Math object (5)
        let num1 = 10.47;
        let num2 = 10.50;
        let num3 = 10.40;
        let num4 = 10.56;
        let num5 = 16;
        let num6 = 2;
        console.log(Math.abs(num1))
        console.log(Math.round(num1))
        console.log(Math.round(num4))
        console.log(Math.ceil(num3))
        console.log(Math.floor(num4))
        console.log(Math.sqrt(num5))
        console.log(Math.sqrt(num6))
        console.log(Math.pow(2, 4))
        console.log(Math.log(10))
```

```
        console.log(Math.max())
        console.log(Math.min())
        // console.log(num1);
        // console.log(num2);
        // console.log(num3);
        // console.log(num4);
        // console.log(num5);
        // console.log(typeof (num1));
        // console.log(typeof (num2));
        // console.log(typeof (num3));
        // console.log(typeof (num4));
        // console.log(typeof (num5));

        // Date Object (5 to 8)

        let myDate = new Date();
        console.log(myDate);
        console.log(myDate.getDate());
        console.log(myDate.getDay());
        console.log(myDate.getMonth());
        console.log(myDate.getFullYear());
        console.log(myDate.getHours());
        console.log(myDate.getMinutes());
        console.log(myDate.getMilliseconds());
        console.log(myDate.getSeconds());
        console.log(myDate.getTime())
        console.log(myDate.toString())
        console.log(myDate.toDateString())

    </script>
```

Operation

String OPerations (5 to 8)

Array Operation (28 ⇒ 15)

```
<script>
        // Operation
        // String OPerations (5 to 8) ==> " ", ' '
        let myName = 'GanapathirajaV';
        let fName = "Ganapathi";
        let mName = "Raja";
```

```
        let lName = "V"
        let fullname = "ganapathirajav";
        let str = "                          gnaapathirajav
";
        let str1 = "ganapathirajav";
        let str2 = 'GANAPATHIARAJAV'
        let str3 = "my Name is Ganpathi i'm mobile app developer";

        console.log(myName);
        console.log(fName);
        console.log(mName);
        console.log(lName);
        console.log(fullname);
        console.log(str);
        console.log(str1);
        console.log(str2);
        console.log(str3);


        console.log(myName.length); // counting the characters

        console.log(fName + " " + mName + lName);
        console.log(fName.concat(mName)); // merge or join
        console.log(fName.concat(mName, lName));
        console.log(lName.concat(fName, mName))

        console.log(fullname.slice(1));
        console.log(fullname.slice(2));
        console.log(fullname.slice(3));
        console.log(fullname.slice(12));

        console.log(fullname.slice(0, 9));
        console.log(fullname.slice(9, 13));

        console.log(fullname.substring(1));
        console.log(fullname.substring(0, 9));

        console.log(fullname.substr(9, 13));

        console.log(str1);
```

```
        console.log(str2);
        console.log(str1.toUpperCase());
        console.log(str2.toLowerCase());

        console.log(str);
        console.log(str.length);
        console.log(str.trim());
        console.log(str.trimEnd());
        console.log(str.trimRight());
        console.log(str.trimStart());
        console.log(str.trimLeft());
        let myfullname = "ganapathi          raja      v";
        console.log(myfullname);
        console.log(myfullname.length);
        console.log(myfullname.trim());

        console.log(str3);
        console.log(str3.replace("mobile", "FullStack"));
        console.log(str3.replace("Mobile", "FullStack"));
        console.log(str3.repeat(5))

        console.log(fullname);
        console.log(fullname.padStart(20));
        console.log(fullname.padEnd(20));

        // Array Operation (28 ⇒ 15 )

 // Array Operation (28 ⇒ 15 )
        const myArray = ["ganapathi", "visal", "vasan", "hari", "akash",
100, 100, 200, 10, 500.60, true, false, '#$%', null, "dog"];
        const myArrayValues1 = [100, 200, 300, 400, 500, 600];
        const myArrayValues2 = [1000, 2000, 3000, 4000, 5000];
        // console.log(myArray);
        // console.log(myArrayValues1);
        // console.log(myArrayValues2);

        // console.log(myArray);
        // console.log(myArray[0]);
        // console.log(myArray[1]);
```

```
        // Common operation
        // push / pop
        console.log(myArrayValues1);
        console.log(myArrayValues1.push(700));
        console.log(myArrayValues1);
        console.log(myArrayValues1.push(800, 900, 1));
        console.log(myArrayValues1);

        console.log(myArrayValues1.pop());
        console.log(myArrayValues1);
        console.log(myArrayValues1.pop())
        console.log(myArrayValues1);
        // unshift / shift
        console.log(myArrayValues2);
        console.log(myArrayValues2.unshift(100, 200, 400));
        console.log(myArrayValues2);

        console.log(myArrayValues2.shift());
        console.log(myArrayValues2);
        console.log(myArrayValues2.shift());
        console.log(myArrayValues2);
        // slice
        console.log(myArray);
        console.log(myArray.slice(1))
        console.log(myArray.slice(2));
        console.log(myArray.slice(2, 5));
        // concat
        console.log(myArrayValues1);
        console.log(myArrayValues2);

        console.log(myArrayValues1 + myArrayValues2);
        console.log(myArrayValues1.concat(myArrayValues2));

// Major Operation
        const myArr = [10, 20, 100, 30, 1000, 1, 2, 5, 0];
        // Map ()
        console.log(myArr);
        function mapArray(x) {
            return x + 10;
        };
```

```
        console.log(myArr.map(mapArray))
        // Sort()
        console.log(myArr);
        console.log("Asc", myArr.sort((a, b) => a - b))
        console.log("Desc", myArr.sort((a, b) => b - a))
        // filter()
        function filterArray(x) {
            return x > 9;
        }
        console.log(myArr.filter(filterArray))
        // find() ==> find the first match
        function findArray(x) {
            return x > 9;
        };
        console.log(myArr.find(findArray));
        // forEach()
        function foreachArray(i, x) {
            return console.log(`find index ${x} and add  values ${i +
100}`);
        }
        console.log(myArr.forEach(foreachArray))
        // reduce()
        function reducerArray(i, x) {
            return i + x;
        };
        const totalvalueofArray = myArr.reduce(reducerArray);
        console.log(totalvalueofArray);
        // include()
        console.log(myArr);
        console.log(myArr.includes(1));
        console.log(myArr.includes(9));
        // indexOf()
        console.log(myArr[1]);
        console.log(myArr.indexOf(1));
        // Array.isArray()
        console.log(typeof (myArr));
        const myArray = () => {
            return hello
        };
        console.log(typeof (myArray));
```

```
        console.log(Array.isArray(myArr));
        console.log(Array.isArray(myArray));
        // reverse
        console.log(myArr.reverse());
        //
        console.log(myArr.findIndex());
        console.log(myArr.length);


        // ... spread ==> coping the array
        // ... rest  ==> getherring remaing values


       // ... spread ==> coping the array
        // ... rest  ==> getherring remaing values

        let myArr1 = [10, 20, 30, 40, 50, 1000];
        let myArr2 = [60, 70, 80, 90, 100];


        console.log(myArr1 + myArr2);

        console.log(myArr1.concat(myArr2));

        let myArr3 = [...myArr1];
        console.log(myArr3.push(2000))
        console.log("My Array 3  : ", myArr3);

        let myArr4 = [...myArr1, ...myArr2]; // spread operator
        console.log(myArr4)


        let [a, b, c, d, ...e] = [1111, 2, 3, 4, 5, 6, 7, 8, 9, 10]; //
rest operator
        console.log(a);
        console.log(b);
        console.log(c);
        console.log(d);
        console.log(e);
    </script>
```

DOM Manipulating(5) & Selecting Elements (5)

Manipulation ⇒ Update
MOdification ⇒ Changes

DOm Manipulating Elements	DOm Selecting Element

Create Elements
Append Elements
Preppend
Remove
textContent
InnerHtml
innerText
Style properties
	document.getElementByTagName();
	document.getElementById();
	document.getElementByclassName();
	document.QuerySelector() //call the first match
	document.QuerySelectorAll()

```
<div>
        <h2>DOM Manipulating & Selecting Elements </h2>

        <h1>hello world</h1>
        <h1 class="hii">hii Team</h1>
        <h1 id="hii">hello team hii friends</h1>
        <h1>Friend & team</h1>
        <h1>hello</h1>
        <style>
            .msg {
                color: blueviolet;
                padding: 20px;
                margin: 20px;
                width: 500px;
                background-color: bisque;
            }
        </style>
        <div id="msg" class="msg"></div>
        <div>
            <div id="idselect"></div>
            <div id="idselect"></div>
            <div id="idselect"></div>
```

```
            <div class="classselect"></div>
            <div class="classselect"></div>
            <div class="classselect"></div>
        </div>
    </div>

    <div>
        <button onclick="nameSelect()">select Tag name </button>
        <button onclick="classSelect()">select Class Name </button>
        <button onclick="idSelector()">select Id Name </button>
        <button onclick="querySelect()">select Query select id </button>
    </div>
    <div>
        <button onclick="createTag()">Create a tag</button>
        <button onclick="myTextmsg()">Click me</button>
    </div>
    <script>
        // DOM Manipulating(5) & Selecting Elements (5)
        function nameSelect() {
            let myText = "hello batch13 guys Good Morning....!!";
            let DisplayText = document.getElementsByTagName("h1");
            DisplayText.textContent = myText
        };

        function classSelect() {
            let myText = "hello batch13 guys Good Morning....!!";
            let DisplayText = document.getElementsByClassName("hii");
            DisplayText.textContent = myText
        }


        const idSelector = () => {
            let myText = "hii friend...👍👌👌👌👌";
            let myIdtag = document.getElementById("hii");
            myIdtag.textContent = myText;
        };


        const createTag = () => {
```

```
            let myTag = document.createElement("h1");
            let myTagp = document.createElement("p");
            let myTagh2 = document.createElement("h2");
            let myTagdiv = document.createElement("div");
            let myTagtext = " Hello Ganapathi ... 😎😎😎😎👌";
            let myTagtext1 = " Hello Ganapathi hellooooo ...
😎😎😎😎👌";
            let myTextdiv = "hii team";

            myTagdiv.style.color = "red";


            myTag.textContent = myTagtext;
            myTagp.innerHTML = myTagtext
            myTagh2.innerText = myTagtext1
            myTagdiv.innerText = myTextdiv



            document.body.append(myTag);
            document.body.append(myTagp);
            document.body.append(myTagh2);
            document.body.append(myTagdiv);

        }

        function myTextmsg() {
            let myText = "hello world hii team....!!!";
            let mySpan = document.createElement("h1");
            mySpan.innerText = myText;

            mySpan.style.color = "green";
            mySpan.style.padding = "20px";
            mySpan.style.margin = "50px";
            mySpan.style.fontSize = "36px";
            mySpan.style.marginTop = "50px";
            mySpan.style.backgroundColor = "black";
            mySpan.style.width = "400px";
```

```
            document.body.append(mySpan);
        }


        let myText = "hello world hii team....!!!";
        let mySpan = document.createElement("h1");
        mySpan.innerText = myText;

        mySpan.style.color = "green";
        mySpan.style.padding = "20px";
        mySpan.style.margin = "50px";
        mySpan.style.fontSize = "36px";
        mySpan.style.marginTop = "50px";
        mySpan.style.backgroundColor = "black";
        mySpan.style.width = "400px";

        document.body.append(mySpan);


        function msg() {
            let myMsg = "good morning ";
            let myTag = document.getElementById("msg");
            myMsg.textContent = myMsg;
        }

        let myMsg = "good morning ";
        let myTag = document.getElementById("msg");
        myTag.textContent = myMsg



        function querySelect() {
            let myMsg = "Hiii visal and Surender.....!!!"
            let myIdselect = document.querySelector("#idselect");
            myIdselect.textContent = myMsg;

        }

    </script>
```

Validation of Static Web Pages

ES 6 Topics

ES 5	Es 6
Variable ⇒ var	Let ,const
Data and Data type ⇒ Primitive data ⇒ number ,string boolean,undegine ,null	BigInt,symbole
Non primitive data type ⇒ array ,object, function Array ⇒ oper Object ⇒ literal Function ⇒ FE	 … spread / rest operator `${}`, this Arrow method ⇒ =()=>{} ⇒ call back
Operator ⇒ 5	Ternary (?)
Looping ⇒ for , while	for(of), for(in)
	async/defer/ throttle
	‘useStrict’,
	Template string ` `
	Asyn / await / try catch ⇒ exceptional handling
	Scheduling ⇒
	Promise Methods ⇒
	Fetching
	Model / module ⇒ import / export
	Default js Programms

Event Handlers
Event handler
Event bubbling
Window Object

Window Object (Contd.)

Some of the commonly used window object properties is as shown below:

Method	Description
innerHeight	Return inner height of the window including scrollbars
innerWidth	Return inner width of the window including scrollbars
closed	Returns a Boolean true if a window is closed
history	Returns the history object for the window
location	Returns the location object for the window
navigator	Returns the navigator object for window
screen	Returns the screen object for window

Window Object (Contd.)

Some of the commonly used window object methods are as shown below:

Method	Description
alert()	Displays an alert box with message and ok button
close()	Closes the current window
confirm()	Displays a dialogue box with a message and ok button
open()	Opens a new browser window
stop()	Stops the window from loading
print()	Prints the current window
prompt()	Displays a dialogue box for getting input from a user

Different Event Handlers

Some of the commonly used event handlers are as shown below:

Event Handler	Description
onclick	when a user clicks the mouse button on a certain HTML element
ondblclick	when a user double clicks the mouse button on a certain HTML element
onmouseover	when a user moves the cursor over a certain element
onmouseout	when a user leaves the cursor from an element
onmousedown	when a user presses a mouse button over an element
onmouseup	when a user releases a mouse button over an element
onmousemove	when a user moves the mouse
onKeyPress	when a user press an alphabetic, numeric or punctuation key

Different Event Handlers (contd.)

Some of the commonly used event handlers are as shown below:

Event handler	Description
onkeydown and onkeyup	when a user has pressed down or released the key (even if that key does not produce a character value)
onfocus	when a user focuses on certain element
onsubmit	when a user submits the form
onchange	when a user modifies the value of a form element
onload	when the browser finished the loading of a page
onunload	when the browser unloads the page

Event Bubbling

- In JavaScript, event bubbling is a method of event propagation in the HTML DOM API.
- It works on the principle that whenever an event happens on an element, it first runs the handlers on it then on its parent, up to the containing elements in the hierarchy.
- Suppose element “X” consists of element “Y”, and element “X” is clicked.
- Then the click event will trigger element “X”, and it will bubble up and trigger it for element “Y” as well.

```
addEventListener(EventType, Action, userCapture)
```

- **`EventType`:** It is a type of event which will trigger the action.
- **`Action`:** It is a task to be performed when the event is triggered.
- **`userCapture`:** It is a Boolean value that indicates the phase of an event. By default, its value is set to false, which represents it is in the bubbling phase.

Event	Description
`onclick`	Fires when an element is clicked
`ondblclick`	Fires when an element is double-clicked
`onmousedown`	Mouse button pressed
`onmouseup`	Mouse button released
`onmousemove`	Mouse pointer moves over an element
`onmouseover`	Mouse enters an element
`onmouseout`	Mouse leaves an element
`oncontextmenu`	Right-click on an element

Event	Description
`onsubmit`	Form submitted
`onreset`	Form reset
`onfocus`	Input field gets focus
`onblur`	Input field loses focus
`onchange`	Value of input, select, or textarea changes
`oninput`	Fires when user types (live input)
`onselect`	Text is selected inside input/textarea

Event	Description
`onload`	Page or element fully loaded
`onunload`	Page is unloaded (refresh/close)
`onresize`	Browser window resized
`onscroll`	Page or element scrolled
`onbeforeunload`	Before page is unloaded (confirmation dialogs)

Menu Toggle Concept Keywords

1. **Element Selection**
    - `document.getElementById()`
    - `document.querySelector()`
2. **Event Handling**
    - `onclick`
    - `addEventListener("click", …)`
3. **Class / Style Manipulation**
    - `element.classList.toggle("className")`
    - `element.classList.add("className")`
    - `element.classList.remove("className")`
    - `element.style.display = "block / none"`
4. **Toggle Logic**
    - `if / else` conditions
    - `menu.classList.contains("show")`
5. **UI Keywords**

- **Hamburger menu (☰)** → open menu
- **Close button (✖)** → close menu
- `hidden / visible`
- `slide-in / slide-out` (for animation concepts)

6. **Responsive Design (CSS + JS combo)**
   - `@media` queries
   - `position: fixed / absolute`
   - `transition` / `transform` (animations for smooth toggle)

Advanced Methods of Js 👍 (ES6)

ES ⇒ ECMA script

⇒ . The full form of ECMA is the European Computer Manufacturers Association

⇒ Js ⇒ java core ⇒ OOPs , script ⇒ js ⇒ before version of Js ⇒ Ecma script

ES 5 ⇒ Old version of js	ES 6 ⇒ new Version of JS
Variables ⇒ Var	Variable ⇒ let,const
Data ⇒ Type ⇒ Primitive / non Primitive	Semi / Reference Data type
Primitive Data type ⇒ string,number,boolean,undefine,null	Bigint, symbole
Non Primitive ⇒ object,array, function ⇒ FE ⇒ function funName(){return }	Arrow method ⇒ call back method Const / let/ funName =()=>{return}
Object ⇒ (.) literal method	Object Destructuring ${object key} .this key word
Array ⇒ common operation (12)	(...) spread / rest Operators
Operators ⇒ 4 (ari,assign,compare,logic)	Ternary (?) ⇒ avoid ⇒ if()else{}
Looping ⇒ for ,while ,do while	for(of),for(in)

'useStrict' ⇒ mandatory use any one fun

```
'use strict' → Strict Mode
'use strict' enables Strict Mode in JavaScript.
Can be used at the beginning of a JavaScript file or inside a function.
'use strict' does not mean "mandatory to use any one function."
```

Template string ⇒ `` ⇒ string interpolation ⇒ back tick

```
It allows us to easily insert variables or expressions into a string.
This is called string interpolation.
Template strings can also create multiple lines easily.
```

async/ defer ⇒ must loading beforely or top

```
 <!-- Extranal scripting -->
    <script src="script.js" async></script>
    <script src="script.js" defer></script>
```

```
async vs defer in <script>
These attributes are used when loading external JavaScript files.
How async works
```

```
The JavaScript file is downloaded while HTML is being parsed.
But once JavaScript finishes downloading, it executes immediately.
Use async when the JavaScript file is independent of the HTML or other
scripts.
defer
defer also downloads JavaScript while HTML is being parsed.
But JavaScript execution waits until the HTML parsing is completed.
```

Scheduling concepts ⇒ Timing scenarios ⇒ setTimeOut(), setTimeIntecal()

```
setTimeout() setTimeout(function, delay);
setTimeout() executes a function only once after a specified delay.
Why?
setTimeout() does not block JavaScript execution.
The browser schedules the function and continues executing the remaining
code.

setInterval() setInterval(function, interval);
setInterval() executes a function repeatedly after every specified
interval.
<script>
        // scheduling Funtion
        // setTimeOut
        // setIntravel

        alert("hii")

        setTimeout(() => { alert("hii teams ") }, 4000)

        setInterval(() => { alert("hii hello team and frineds...! ") },
8000)

        // js time seconds convert
        // 1000 millsecond
        // 5 * 1000 // seconds
        // 5 * 60 * 1000 // minu
    </script>
```

Async / await / try /catch ⇒exception handling

Promise Methods ⇒ resolve,reject,pending

```
<script>
        // let voteAge = 18;
        let voteAge = 17;
        if (voteAge >= 18) {
            console.log("Eligible to vote....!!");
        }
        else {
            console.log("NOt Eligible to Vote...!!!")
        }

        const myVote = new Promise((resolve, reject) => {
            // let voteAge = 17;
            // let voteAge = 27;
            // let voteAge = "18";
            if (voteAge >= 18) {
                resolve("Eligible to vote....!!");
            }
            else {
                reject("NOt Eligible to Vote...!!!")
            }
        });
        console.log(myVote);
    </script>
```

Fetch ⇒ Getting the data from out source

```
fetch() → Getting data from an external source
fetch() is a JavaScript function used to request data from an external
source, usually an API.
 <div id="usersdata">

    </div>

    <script>
        // Fetching
        // fetch("url")
        // .then((res)=>{format})
        // .then(((data) =>{data}))
        // .catch((err)=>{err})
```

```
        fetch("https://jsonplaceholder.typicode.com/users")
            .then((hi) => hi.json())
            .then((data) => { console.log(data) })
            .catch((err) => { console.log(err) })



        // fetch("https://jsonplaceholder.typicode.com/users")
        //     .then((res) => res.json())
        //     .then((data) => {
        //         console.log(data)

        //         let myData = ''

        //         myData += data.map((u) => {
        //             `<p ${u.id}> ${u.name} </p>`
        //             document.getElementById("usersdata").innerHTML =
data
        //         })

        //     })
        //     .catch((err) => { console.log(err) })
    </script>
```

Model / Module ⇒ import / export

```
Model / Module → import / export
A module is a JavaScript file containing code that can be shared with
other JavaScript files.
import
import allows us to use an exported value from another module.
Default Export
We can export one main value as the default export.
// es 6
import decValues from './deccount'
import addCount from './addcount'
```

```
// es 5
const addValue = require('./addcount');
const decCount = require("./deccount");


function overall() {
    let mycount = document.getElementById("count");
    let Count = 0;

    function updateCOunt() {
        mycount.textContent = Count;
    }
}

export default overall;
```

Default Programmings

```
    <div id="usersdata">

    </div>
    <div id="usersdata1">

    </div>
    <script>
        // Es 6 Concepts
        //  aritnemeticc

        // let a = 100, b = 200;
        // function add() {
        //     const c = a + b;
        //     // return c;
        //     console.log(c);
        // }
        // add()
        // function add() {
```

```
        //     const c = a + b;
        //     // return c;
        //     console.log(c);
        // }
        // add()
        // // `useStrict`
        // function sub() {
        //     const c = a - b;
        //     // return c;
        //     console.log(c);
        // }
        // sub()
        // // `useStrict`
        // function multi() {
        //     const c = a * b;
        //     // return c;
        //     console.log(c);
        // }
        // multi()

        // const myName = {
        //     name: "ganpathi",
        //     gmail: "ganapathi@gmail.com",
        //     mobile: 7502132895
        // };
        // console.log(myName);



        // console.log("My name is ", myName.name, " and my mobile number
is ", myName.mobile);
        // console.log("My name is ",
        //     myName.name, " and my mobile number is ",
        //     myName.mobile);

        // console.log(`My name is ", ${myName.name} ,
        //  " and my mobile number is ",
        //   ${myName.mobile}`) // template string  or string
interponational
```

```
        // fetching
        fetch("https://jsonplaceholder.typicode.com/users")
            // fetch("./data.json")
            .then((res) => res.json())
            .then((data) => {
                console.log(data)
                let useData = '';
                data.map((u) => {
                    useData += `
                        <p ${u.id}> ${u.name} </p>
                    `
                    document.getElementById("usersdata").innerHTML =
useData;
                })
            })
            .catch((error) => { console.log(error) })

        // fetch("https://jsonplaceholder.typicode.com/todos")
        //     .then((res) => res.json())
        //     .then((data) => {
        //         console.log(data)
        //         let useData1 = '';
        //         data.map((u) => {
        //             useData1 += `
        //                 <h1 ${u.id}> ${u.title} </h1>
        //             `
        //             document.getElementById("usersdata1").innerHTML =
useData1;
        //         })
        //     })
        //     .catch((error) => { console.log(error) })


        // scheduling concepts
        // setTimeout()
        // setIntravel()
        console.log('hii team.....!!')
        setTimeout(() => { console.log('hii team.....!!') }, 5000);
        setInterval(() => { console.log('hii team......join ur
session..!!') }, 8000);
```

```
        // Promise method
        // let myAge = 19;
        // if (myAge >= 18) {
        //     console.log("Eligible");
        // }
        // else {
        //     console.log("Not Eligible");
        // }
        // let myVoting = new Promise((resolve, reject) => {
        //     let myAge = 19;
        //     if (myAge >= 18) {
        //         // console.log("Eligible");
        //         resolve("Eligible");
        //     }
        //     else {
        //         // console.log("Not Eligible");
        //         reject("Not Eligible");
        //     }
        // });
        // console.log(myVoting);

        // await ==>
        // wait  ==>
        // sync ==>
        // async ==>

        // otp
        let otpGen = () => {
            const otpNumber = 1234;
            // const otpNumber = false;
            return new Promise((resolve, reject) => {
                if (otpNumber === 1234) {
                    resolve("Otp sended successfully ... valid for next 5
second ", otpNumber)
                }
                else {
                    reject("Otp not send...!");
                }
            });
```

```
}

// otpGen();
// console.log(otpGen());

let otpRecive = () => {
    setTimeout(() => {
        if (otpGen()) {
            console.log("otp reviced..", otpGen())
        }
        else {
            console.log("otp not recived")
        }
    }, 5000);
};

// otpRecive();
// console.log(otpRecive());

async function otpVerification() {
    try {
        // await otpRecive();
        let myOtpVerify = await otpRecive();
        console.log(myOtpVerify);
    }
    catch (err) {
        console.error(err);
    }
    // finally {
    //     console.log("All process done ...!!")
    // }
};
console.log(otpGen())
console.log(otpRecive())
console.log(otpVerification())


// localStorage.setItem();
// localStorage.getItem();
// localStorage.removeItem();
```

```
        // localStorage.clear

        // sessionStorage.setItem();
        // sessionStorage.getItem();
        // sessionStorage.removeItem()

        // document.cookie()

        let myDataFetcing = async () => {
            try {
                await setTimeout(() => {
                    fetch("https://jsonplaceholder.typicode.com/users")
                        // fetch("./data.json")
                        .then((res) => res.json())
                        .then((data) => {
                            console.log(data)
                            let useData = '';
                            data.map((u) => {
                                useData += `
                            <p ${u.id}> ${u.name} </p>
                    `

document.getElementById("usersdata").innerHTML = useData;
                            })
                        })
                        .catch((error) => { console.log(error) })
                }, 8000)
            }
            catch (err) {
                console.log("feching is not occur")
            }
        };

        console.log(myDataFetcing());



    </script>
```

```
    <!-- asyn / defer / throttal  ==> enhence the code or project
productivity ==> loading   -->
    <!--
    <script src="counterapp.js" async></script>
    <script src="counterapp.js" defer></script> -->
```